Camping World PPT Generator — Final Implementation Plan

Date: April 3, 2026
Version: 4.0 (LangGraph Edition)

Table of Contents

1. System Overview
2. Finalized Tech Stack
3. Architecture & Pipeline
4. Project Structure
5. Template Parser Design
6. LangGraph Pipeline Design
7. Pipeline Tools
8. Data Models & Schemas
9. API Design
10. User Journeys
11. Phase-by-Phase Implementation
12. Cost Estimation
13. Risks & Mitigations

---

1. System Overview

What the System Does

An agentic system that takes a Camping World master PPTX template + a natural language prompt and produces a brand-consistent, presentation-ready PPTX **in one shot with 99% accuracy**.

Core Principles

- **Template is truth:** Generated PPTX is the original template with content filled in. Layout, fonts, colors, images — all inherited from the template itself.
- **Images are never touched:** Logos, photos, decorative images remain exactly as in the template.

- **Charts keep their styling:** Only data values change. Chart type, colors, gridlines, legends — all preserved.
- **LLM decides slide count:** Template may have 30 layouts, but the prompt might only need 10. The LLM picks the right number and layouts.
- **Guidance slides are read, not used:** Some templates contain instruction/style-guide slides. The system reads them as rules, then excludes them from generation.
- **Deterministic rendering:** Only 2-3 LLM calls in the pipeline. Everything else (parsing, validation, rendering) is pure Python.

Two User Journeys

1. **Upload + Prompt:** User uploads a new PPTX template + provides a prompt → system auto-profiles the template → generates PPT
2. **Select + Prompt:** User selects a previously uploaded/profiled template from library → provides prompt → generates PPT

---

2. Finalized Tech Stack

Layer	Choice	Why
Backend	FastAPI + BackgroundTasks	Native async, python-pptx ecosystem, simple — no Redis/Celery
Async DB Driver	Motor	Async MongoDB driver, connection pooling
Database	MongoDB (collections + GridFS)	Flexible document schema for template profiles; GridFS for PPTX binary storage
Pipeline Orchestration	LangGraph v1.1.4	Explicit async StateGraph, typed state, conditional edges, multi-model per node
Content LLM	Claude Opus 4.6	Best reasoning for content planning within template constraints
Guidance LLM	Claude Sonnet 4.6	Fast, cheap — guidance extraction, slide classification
LLM Integration	LangChain (core + providers)	`init_chat_model()` for provider-agnostic multi-model support
PPTX Engine	python-pptx + lxml	Template-aware rendering; lxml for deep XML parsing beyond python-pptx API
Validation	Pure Python (deterministic)	No LLM — check text fits, chart data valid, required fields present
Frontend	Next.js (later phase)	SSR, file uploads, SSE support

Explicitly NOT Using (Phase 1)

- Docker
- Redis / Celery
- S3 / MinIO
- LibreOffice
- Vision QA
- Anthropic Agent SDK / CrewAI
- AI image generation

---

3. Architecture & Pipeline

High-Level Architecture

```
┌──────────────────────────────────────────────────────────────────┐
│                     Next.js Frontend (Phase 2)                   │
│   Upload Template  |  Select Template  |  Prompt  |  Download|
└──────────────────────────────┬───────────────────────────────────┘
                               │
┌──────────────────────────────▼───────────────────────────────────┐
│                      FastAPI Backend                              |
│                                                                   │
│  POST /api/templates/upload  → Template Analysis Pipeline         │
│  GET  /api/templates         → List profiled templates            │
│  POST /api/generate          → Generation Pipeline                │
│  GET  /api/generate/{id}/status   → SSE progress stream          |
│  GET  /api/generate/{id}/download → Download PPTX                |
└──────────────────────────────┬───────────────────────────────────┘
                               │
┌──────────────────────────────▼───────────────────────────────────┐
│          LANGGRAPH ASYNC PIPELINE (StateGraph)                    │
│                                                                   │
│  PipelineState (TypedDict) flows through each node:               │
│                                                                   │
│  load_profile ──► plan_outline (Opus) ──► plan_content (Opus)|
│       |                                          |                |
│       |                ┌──────────────────────────┘                │
│       |                ▼                                          |
│       |          validate ──┬── valid ──► render ──► store        │
```

```
│       │                    └── invalid ──▶ plan_content      │
│       │                        (max 2 retries)               │
│                                                              │
│  Each node = async Python function                           │
│  LLM calls via init_chat_model() (multi-model per node)     │
│  Pure Python nodes: validate, render, store                  │
│  SSE progress via stream_mode="custom"                       │
└──────────────────────────┬──────────────────────────────────┘
                           │
┌──────────────────────────▼──────────────────────────────────┐
│                      MongoDB                                 │
│                                                              │
│  Collections:                                                │
│    template_profiles  →  profile JSON + brand rules          │
│    generations        →  prompt, slide plan, status          │
│                                                              │
│  GridFS Buckets:                                             │
│    template_files     →  original PPTX binaries              │
│    generated_files    →  output PPTX binaries                │
└──────────────────────────────────────────────────────────────┘
```

Generation Pipeline (per request)

```
PROMPT + TEMPLATE
       │
       ▼
┌──────────────────────────────────────────────────────────┐
│  1. TEMPLATE PARSER (deterministic + LLM assist)         │
│     (runs once per template, cached after)               │
│                                                          │
│  A. Structural Parse (python-pptx + lxml):               │
│     ├─ Extract ALL slides                                │
│     ├─ Per slide: placeholders, charts, images,          │
│     │   positions, fonts, theme                          │
│     └─ Output: raw slide metadata                        │
│                                                          │
│  B. Guidance Extractor (Claude Sonnet):                  │
│     ├─ Read text content of ALL slides                   │
│     ├─ Classify each slide:                              │
│     │   "guidance" vs "content_layout"                   │
│     ├─ From guidance slides extract:                     │
```

```
│     │   ├─ Font rules (heading: 36pt bold, etc.)   │
│     │   ├─ Color usage rules                       │
│     │   ├─ Tone/voice guidelines                   │
│     │   ├─ Formatting dos/don'ts                   │
│     │   └─ Any other brand instructions            │
│     └─ Output: brand_rules JSON +                  │
│        filtered layout list (guidance excluded)     │
│                                                    │
│  Combined Output: Template Profile JSON            │
│    ├─ available_layouts[] (guidance slides removed) │
│    ├─ brand_rules{} (extracted from guidance)      │
│    ├─ chart_metadata per layout                    │
│    └─ theme{} (colors, fonts from XML)             │
└────────────────────┬──────────────────────────────┘
                     ▼

┌──────────────────────────────────────────────────┐
│  2. CONTENT PLANNER (Claude Opus) — TWO PHASES    │
│                                                    │
│  Input: prompt + Template Profile + brand_rules    │
│                                                    │
│  Phase A — OUTLINE:                                │
│     ├─ Analyze prompt scope and complexity         │
│     ├─ Decide optimal slide count (e.g., 12)      │
│     ├─ Define narrative arc                        │
│     │   (intro → problem → solution → data → CTA)  │
│     ├─ For each slide in arc:                      │
│     │   pick best layout from available_layouts    │
│     └─ Output: slide_outline[]                     │
│        [{layout_index, purpose, content_brief}]    │
│                                                    │
│  Phase B — POPULATE:                               │
│     ├─ For each slide in outline:                  │
│     │   ├─ Generate text per placeholder           │
│     │   │   RESPECTING brand_rules                 │
│     │   │   (tone, style, formatting instructions) │
│     │   ├─ Generate chart data if chart exists     │
│     │   │   (matching series/category structure)   │
│     │   └─ Images: skip (keep from template)       │
│     └─ Output: Slide Plan JSON                     │
└────────────────────┬──────────────────────────────┘
                     ▼
```

```
┌──────────────────────────────────────────────────────┐
│  3. VALIDATOR (pure Python, no LLM)                  │
│     ├─ Text length ≤ max_chars per placeholder       │
│     ├─ Chart data matches structure                  │
│     │   (correct # series/categories,                │
│     │    pie chart sums to 100, etc.)                │
│     ├─ Slide count matches outline                   │
│     ├─ No duplicate layout variants used             │
│     ├─ All required placeholders filled              │
│     └─ If invalid → retry Planner Phase B            │
│        with specific error messages (max 2x)         │
└──────────────────────┬───────────────────────────────┘
                       ▼ (valid)

┌──────────────────────────────────────────────────────┐
│  4. RENDERER (python-pptx, no LLM)                   │
│     ├─ Open original template PPTX                   │
│     ├─ Keep ONLY slides selected in outline          │
│     ├─ DELETE all other slides (guidance +           │
│     │   unused layouts)                              │
│     ├─ For each kept slide:                          │
│     │   ├─ Find text placeholders                    │
│     │   │   → fill with generated text               │
│     │   │   → PRESERVE original font, size,          │
│     │   │     color, bold/italic                     │
│     │   ├─ Find charts                               │
│     │   │   → replace data values only               │
│     │   │   → keep chart type, colors, styling       │
│     │   ├─ Images → DON'T TOUCH                      │
│     │   └─ Shapes/decorations → as-is                │
│     ├─ Clean placeholder text                        │
│     │   ("INSERT TEXT HERE", "LOREM", etc.)          │
│     └─ Save final PPTX → GridFS                      │
└──────────────────────┬───────────────────────────────┘
                       ▼
          PPTX READY ✓
```

Why 99% Accuracy in One Go

Aspect	How It's Guaranteed
Layout fidelity	Original template file used — layouts ARE the template

Aspect	How It's Guaranteed
Font consistency	python-pptx captures and preserves original text formatting before replacing
Color consistency	Theme colors inherited from template; no custom RGB ever set
Chart styling	Only `chart.replace_data()` — chart type, colors, gridlines, legends preserved
Images	Never touched — logos, photos remain exactly as in template
Content fit	Parser estimates max chars per placeholder → Planner generates within limits → Validator catches overflows
Chart data integrity	Parser extracts exact series/category structure → Planner must match → Validator verifies
Brand voice/style	Guidance slides read and fed as rules to the Planner
Slide count	LLM decides optimal count based on prompt complexity; only relevant layouts selected

---

4. Project Structure

```
PPT/
├── backend/
│   ├── app/
│   │   ├── __init__.py
│   │   ├── main.py                    # FastAPI app factory, lifespan
(startup/shutdown)
│   │   ├── config.py                  # pydantic-settings: env vars,
defaults
│   │   ├── database.py                # Motor async client, GridFS setup
│   │   |
│   │   ├── api/
│   │   │   ├── __init__.py
│   │   │   ├── dependencies.py        # Dependency injection (get_db,
get_gridfs)
│   │   │   ├── middleware.py          # CORS, request logging, error
handling
│   │   │   └── routes/
│   │   │       ├── __init__.py
│   │   │       ├── templates.py       # POST /upload, GET /list, GET
/{id}
```

```
|   |   |       ├── generate.py        # POST /generate, GET /{id}/status,
GET /{id}/download
|   |   |       └── health.py          # GET /health
|   |   |
|   |   ├── models/                    # Pydantic request/response models
(API layer)
|   |   |   ├── __init__.py
|   |   |   ├── template.py            # TemplateUploadResponse,
TemplateListItem
|   |   |   ├── generation.py          # GenerateRequest,
GenerateResponse, StatusResponse
|   |   |   └── common.py             # Shared models (Pagination,
ErrorResponse)
|   |   |
|   |   ├── schemas/                   # Internal data contracts
|   |   |   ├── __init__.py
|   |   |   ├── template_profile.py    # TemplateProfile, SlideLayout,
Placeholder, ChartMeta, BrandRules
|   |   |   └── slide_plan.py          # SlidePlan, SlideOutline,
SlideContent
|   |   |
|   |   ├── services/                  # Business logic layer
|   |   |   ├── __init__.py
|   |   |   ├── template_service.py    # Upload, parse, store profile
|   |   |   └── generation_service.py  # Orchestrate full generation
pipeline
|   |   |
|   |   ├── graph/                     # LangGraph pipeline definition
|   |   |   ├── __init__.py
|   |   |   ├── pipeline.py            # StateGraph definition, node
wiring, conditional edges
|   |   |   ├── state.py               # PipelineState TypedDict +
TemplateAnalysisState
|   |   |   ├── nodes.py               # All node functions (load, plan,
validate, render, store)
|   |   |   └── prompts/               # System prompts as text files
|   |   |       ├── planner_outline.txt
|   |   |       ├── planner_populate.txt
|   |   |       └── guidance_extractor.txt
|   |   |
|   |   ├── tools/                     # Deterministic tools (called by
```

```
graph nodes)
│   │   │   ├── __init__.py
│   │   │   ├── template_parser.py     # python-pptx + lxml: extract
structure
│   │   │   ├── guidance_extractor.py  # LLM classifies guidance vs layout
slides
│   │   │   ├── validator.py           # Pure Python: validate slide plan
against profile
│   │   │   ├── renderer.py            # python-pptx: fill placeholders,
update chart data
│   │   │   └── storage.py             # GridFS upload/download helpers
│   │   │
│   │   └── core/
│   │       ├── __init__.py
│   │       ├── exceptions.py          # TemplateParseError,
ValidationError, RenderError
│   │       └── logging.py            # Structured JSON logging setup
│   │
│   ├── tests/
│   │   ├── __init__.py
│   │   ├── conftest.py               # Fixtures: test DB, sample
templates
│   │   ├── test_template_parser.py
│   │   ├── test_validator.py
│   │   ├── test_renderer.py
│   │   └── test_api/
│   │       ├── test_templates.py
│   │       └── test_generate.py
│   │
│   ├── requirements.txt
│   ├── .env.example
│   └── pyproject.toml
│
└── frontend/                          # Next.js (Phase 2)
    ├── app/
    │   ├── page.tsx                   # Home / template gallery
    │   ├── upload/page.tsx            # Upload template
    │   └── generate/page.tsx          # Prompt + generate + download
    ├── components/
    └── package.json
```

Structure Design Decisions

Pattern	Why
`api/routes/` separate from `services/`	Routes handle HTTP; services contain business logic — testable independently
`models/` vs `schemas/`	`models/` = API request/response. `schemas/` = internal contracts (template profile, slide plan)
`graph/prompts/` as `.txt` files	Prompts are long, change frequently — easy to iterate without touching Python
`graph/` with StateGraph pipeline	Explicit node-based orchestration; each node independently testable; typed state flows through
`tools/` as standalone modules	Each tool independently testable; called directly by graph nodes
`database.py` with Motor	Single async connection, pooled, shared via app lifespan

---

5. Template Parser Design

What Gets Extracted

```
# Per slide in the template:
{
    "slide_index": 5,
    "classification": "content_layout",   # or "guidance" — set by LLM

    # Text placeholders
    "placeholders": [
        {
            "idx": 0,
            "name": "Title 1",
            "type": "TITLE",                # TITLE, SUBTITLE, BODY, CENT
            "position": {
                "left_emu": 457200,         # EMUs (1 inch = 914400 EMU)
                "top_emu": 274638,
                "width_emu": 8229600,
                "height_emu": 1143000
            },
            "text_style": {
                "font_family": "Montserrat ExtraBold",
```

```
                "font_size_pt": 36,
                "bold": True,
                "italic": False,
                "color": "scheme:dk1"       # theme reference, not hardcc
            },
            "max_chars_estimate": 52,       # estimated from width ÷ avg
            "current_text": "INSERT TITLE HERE"  # existing placeholder t
        }
    ],

    # Charts
    "charts": [
        {
            "shape_index": 3,
            "chart_type": "BAR_CLUSTERED",  # BAR, PIE, LINE, etc.
            "series_count": 2,
            "category_count": 4,
            "categories": ["Q1", "Q2", "Q3", "Q4"],
            "series_names": ["Revenue", "Cost"],
            "current_values": [[100, 120, 140, 160], [80, 90, 100, 110]],
            "color_scheme": ["#319D4F", "#E42A2A"]
        }
    ],

    # Images (flagged as DO NOT TOUCH)
    "images": [
        {
            "shape_index": 7,
            "position": {"left_emu": 0, "top_emu": 0, "width_emu": 200000
            "description": "logo"   # inferred from position/size
        }
    ],

    # Tables
    "tables": [
        {
            "shape_index": 4,
            "rows": 5,
            "cols": 3,
            "header_row": ["Metric", "Q1", "Q2"],
            "cell_style": {"font_family": "Overpass", "font_size_pt": 12}
        }
    ]
}
```

Theme Extraction (from `ppt/theme/theme1.xml` via lxml)


```
{
    "colors": {
        "dk1": "#000000",
        "dk2": "#44546A",
        "lt1": "#FFFFFF",
        "lt2": "#E7E6E6",
        "accent1": "#319D4F",
        "accent2": "#E42A2A",
        "accent3": "#CBCBCB",
        "accent4": "#1A6B30",
        "accent5": "#F25555",
        "accent6": "#70AD47",
        "hyperlink": "#0563C1",
        "followedHyperlink": "#954F72"
    },
    "fonts": {
        "major_font": "Montserrat ExtraBold",    # headings
        "minor_font": "Overpass"                   # body text
    }
}
```


Brand Rules Extraction (from guidance slides via LLM)


```
{
    "tone": "Professional, confident, outdoor lifestyle",
    "font_rules": {
        "headings": "Montserrat ExtraBold, 36pt, uppercase on divider sli
        "body": "Overpass Regular, 14-18pt",
        "bullets": "Overpass, 16pt, max 5 bullet points per slide"
    },
    "color_rules": "Primary green (#319D4F) for headers, red (#E42A2A) fo
    "formatting": [
        "Keep slide titles under 6 words",
        "Bullet points should be concise phrases, not full sentences",
        "Use Title Case for headings",
        "Max 5 data points per chart for readability"
    ],
    "restrictions": [
        "Do not use clip art",
```

```
        "Do not change the logo placement",
        "Always include the tagline on the closing slide"
    ]
}
```

---

6. LangGraph Pipeline Design

6.1 Pipeline State

All data flows through a single typed state object, passed between nodes.

```
from typing import TypedDict, Literal
from app.schemas.template_profile import TemplateProfile
from app.schemas.slide_plan import SlidePlan

class PipelineState(TypedDict, total=False):
    # Inputs
    template_id: str
    prompt: str
    generation_id: str

    # Populated by nodes
    profile: TemplateProfile            # From load_profile node
    slide_outline: list[dict]           # From plan_outline node
    slide_plan: SlidePlan               # From plan_content node
    validation_result: dict             # From validate node
    retry_count: int                    # Tracks validation retries (max
    pptx_bytes: bytes                   # From render node
    output_file_id: str                 # From store node

    # Pipeline control
    status: Literal["running", "completed", "failed"]
    error: str | None
```

6.2 Graph Definition (StateGraph)

```
from langgraph.graph import StateGraph, START, END

builder = StateGraph(PipelineState)
```

```
# Register nodes
builder.add_node("load_profile", load_profile_node)
builder.add_node("plan_outline", plan_outline_node)
builder.add_node("plan_content", plan_content_node)
builder.add_node("validate", validate_node)
builder.add_node("render", render_node)
builder.add_node("store", store_node)

# Wire edges
builder.add_edge(START, "load_profile")
builder.add_edge("load_profile", "plan_outline")
builder.add_edge("plan_outline", "plan_content")
builder.add_edge("plan_content", "validate")

# Conditional routing: validate → render (if valid) or plan_content (if i
def validation_router(state: PipelineState) -> str:
    result = state["validation_result"]
    if result["valid"]:
        return "render"
    if state.get("retry_count", 0) >= 2:
        return "render"   # proceed with warnings after max retries
    return "plan_content"

builder.add_conditional_edges("validate", validation_router, ["render", "
builder.add_edge("render", "store")
builder.add_edge("store", END)

# Compile
pipeline = builder.compile()
```

6.3 Node Functions

Each node is an async function that receives `PipelineState` and returns a partial state update.

load_profile — Pure Python (MongoDB read)

```
async def load_profile_node(state: PipelineState) -> dict:
    profile = await template_service.get_profile(state["template_id"])
    return {"profile": profile}
```

plan_outline — LLM call (Claude Opus 4.6)

```
from langchain_core.language_models import init_chat_model

async def plan_outline_node(state: PipelineState) -> dict:
    llm = init_chat_model("claude-opus-4-6", model_provider="anthropic")
    prompt = load_prompt("planner_outline.txt").format(
        profile=state["profile"],
        user_prompt=state["prompt"]
    )
    response = await llm.ainvoke([
        SystemMessage(content=OUTLINE_SYSTEM_PROMPT),
        HumanMessage(content=prompt)
    ])
    outline = parse_json(response.content)
    return {"slide_outline": outline}
```

plan_content — LLM call (Claude Opus 4.6)

```
async def plan_content_node(state: PipelineState) -> dict:
    llm = init_chat_model("claude-opus-4-6", model_provider="anthropic")

    # Include validation errors if this is a retry
    errors_context = ""
    if state.get("validation_result") and not state["validation_result"][
        errors_context = f"\n\nFix these errors:\n{state['validation_resu

    prompt = load_prompt("planner_populate.txt").format(
        profile=state["profile"],
        outline=state["slide_outline"],
        errors=errors_context
    )
    response = await llm.ainvoke([
        SystemMessage(content=POPULATE_SYSTEM_PROMPT),
        HumanMessage(content=prompt)
    ])
    plan = SlidePlan.model_validate_json(response.content)
    return {
        "slide_plan": plan,
        "retry_count": state.get("retry_count", 0) + (1 if errors_context
    }
```

validate — Pure Python (no LLM)

```
async def validate_node(state: PipelineState) -> dict:
    result = validate_slide_plan(state["slide_plan"], state["profile"])
    return {"validation_result": result}
```

render — Pure Python (python-pptx)

```
async def render_node(state: PipelineState) -> dict:
    pptx_bytes = await render_pptx(state["slide_plan"], state["template_i
    return {"pptx_bytes": pptx_bytes}
```

store — Pure Python (GridFS write)

```
async def store_node(state: PipelineState) -> dict:
    file_id = await storage.store_generated_pptx(
        state["pptx_bytes"], state["generation_id"]
    )
    return {"output_file_id": file_id, "status": "completed"}
```

6.4 Multi-Model Support

Each node can use a different LLM provider via `init_chat_model()` :

```
# Claude Opus for content planning (best reasoning)
llm = init_chat_model("claude-opus-4-6", model_provider="anthropic")

# Claude Sonnet for guidance extraction (fast, cheap)
llm = init_chat_model("claude-sonnet-4-6", model_provider="anthropic")

# Google Gemini (optional, future swap)
llm = init_chat_model("gemini-2.0-flash", model_provider="google_genai")

# OpenAI GPT-4o (optional, future swap)
llm = init_chat_model("gpt-4o", model_provider="openai")
```

Model selection is configured per node in `config.py` — no code changes needed to swap providers.

6.5 SSE Progress Streaming

Using LangGraph's `stream_mode="custom"` to emit progress events via SSE:

```
from langgraph.config import get_stream_writer

async def plan_outline_node(state: PipelineState) -> dict:
    writer = get_stream_writer()
    writer({"stage": "planning_outline", "message": "Planning slide struc
    # ... LLM call ...
    return {"slide_outline": outline}

# In generation service:
async def stream_generation(generation_id: str, template_id: str, prompt:
    async for chunk in pipeline.astream(
        {"template_id": template_id, "prompt": prompt, "generation_id": g
        stream_mode="custom"
    ):
        yield f"event: progress\ndata: {json.dumps(chunk)}\n\n"
```

6.6 Content Planner Detail

Model: Claude Opus 4.6
Role: Takes template profile + user prompt → produces a complete, validated slide plan in two phases.

Phase A — Outline (plan_outline node):

```
1. Determine optimal slide count based on prompt complexity:
   - Simple pitch: 5-8 slides
   - Standard review: 10-15 slides
   - Detailed analysis: 15-25 slides
2. Define narrative arc (intro → problem → solution → evidence → CTA)
3. For each slide, select the best layout from available_layouts
4. Output: slide outline as structured JSON
```

Phase B — Populate (plan_content node):

```
1. For each slide in outline: generate text within max_chars limits
2. Follow all brand_rules (tone, formatting, restrictions)
3. For charts: generate realistic data matching exact series/category
structure
4. For tables: generate data matching column count
5. DO NOT generate content for image placeholders
```

6.7 Template Analyzer (Runs Once at Upload)

Model: Claude Sonnet 4.6
Role: Parses uploaded PPTX, classifies guidance vs content slides, extracts brand rules.

This is a separate, simpler graph (or direct function call) since it runs once per template upload, not per generation. It uses the guidance extractor tool which calls Sonnet to classify slides.

---

7. Pipeline Tools (called directly by graph nodes)

All tools are regular async Python functions in the `tools/` directory — no MCP server needed. Graph nodes import and call them directly.

7.1 Template Tools

Tool	Input	Output	Implementation
`parse_template`	`{pptx_path: str}`	Full structural analysis JSON	python-pptx + lxml: slides, placeholders, charts, images, tables, theme
`extract_guidance`	`{raw_slides: list, all_text: dict}`	`{brand_rules: {}, classified_slides: []}`	Claude Sonnet classifies guidance vs content, extracts rules
`save_template_profile`	`{profile: TemplateProfile}`	`{template_id: str}`	Motor write to `template_profiles` collection
`load_template_profile`	`{template_id: str}`	`{profile: TemplateProfile}`	Motor read from `template_profiles` collection

7.2 Validation & Rendering Tools

Tool	Input	Output	Implementation
`validate_slide_plan`	`{plan: SlidePlan, profile: TemplateProfile}`	`{valid: bool, errors: [], warnings: []}`	Pure Python: char limits, chart structure, required fields, no duplicate variants
`render_pptx`	`{plan: SlidePlan, template_id: str}`	`{pptx_bytes: bytes, slide_count: int}`	python-pptx: copy template, keep only selected slides, fill placeholders preserving formatting, update chart data, clean boilerplate text

7.3 Storage Tools

Tool	Input	Output	Implementation
`store_file`	`{data: bytes, filename: str, bucket: str}`	`{file_id: str}`	Motor GridFS upload
`retrieve_file`	`{file_id: str, bucket: str}`	`{data: bytes}`	Motor GridFS download
`save generation`	`{template id, prompt, plan, file_id}`	`{generation_id: str}`	Motor write to `generations` collection

---

8. Data Models & Schemas

8.1 Template Profile (MongoDB document)

```
{
    "_id": "ObjectId",
    "name": "CWGS Standard Q1 2025",
    "template_file_id": "GridFS ObjectId",
    "theme": {
        "colors": {
            "dk1": "#000000", "lt1": "#FFFFFF",
            "accent1": "#319D4F", "accent2": "#E42A2A",
            "accent3": "#CBCBCB", "accent4": "#1A6B30",
            "accent5": "#F25555", "accent6": "#70AD47"
        },
        "fonts": {
            "major_font": "Montserrat ExtraBold",
            "minor_font": "Overpass"
        }
    },
    "brand_rules": {
        "tone": "Professional, confident, outdoor lifestyle",
        "font_rules": { ... },
        "color_rules": "...",
        "formatting": [ ... ],
        "restrictions": [ ... ]
    },
    "slides": [
        {
```

```
            "index": 8,
            "classification": "content_layout",
            "slide_type": "title",
            "placeholders": [
                {
                    "idx": 0, "name": "Title 1", "type": "TITLE",
                    "max_chars": 52,
                    "text_style": {
                        "font_family": "Montserrat ExtraBold",
                        "font_size_pt": 36, "bold": true, "color": "schem
                    },
                    "position": { "left_emu": 457200, "top_emu": 274638,
                }
            ],
            "charts": [],
            "images": [{ "shape_index": 7, "description": "logo" }],
            "tables": []
        }
    ],
    "slide_type_pools": {
        "title": [8, 9],
        "divider": [14, 15],
        "bullet": [23, 24, 25],
        "chart_bar": [49, 50],
        "chart_pie": [52],
        "thank_you": [60]
    },
    "guidance_slide_indices": [0, 1, 2, 3, 4],
    "total_slides": 64,
    "created_at": "2026-04-03T10:00:00Z"
}
```


8.2 Slide Plan (Generated by Content Planner)


```
{
  "metadata": {
    "prompt": "Quarterly business review for Good Sam RV Insurance",
    "decided_slide_count": 12,
    "narrative_arc": "title → agenda → market_overview → performance → me
  },
  "slides": [
    {
```

```
    "slide_type": "title",
    "template_slide_index": 8,
    "purpose": "Opening title slide",
    "content": {
      "placeholders": {
        "0": "Good Sam RV Insurance",
        "1": "Q1 2026 Business Review"
      }
    }
  },
  {
    "slide_type": "chart_bar",
    "template_slide_index": 49,
    "purpose": "Revenue comparison by quarter",
    "content": {
      "placeholders": {
        "0": "Revenue Growth"
      },
      "chart_data": {
        "categories": ["Q1", "Q2", "Q3", "Q4"],
        "series": [
          { "name": "2025", "values": [38.2, 41.5, 44.1, 47.3] },
          { "name": "2026", "values": [42.3, null, null, null] }
        ]
      }
    }
  },
  {
    "slide_type": "bullet",
    "template_slide_index": 23,
    "purpose": "Key performance highlights",
    "content": {
      "placeholders": {
        "0": "Q1 Performance Highlights",
        "1": "QUARTERLY REVIEW",
        "2": [
          "Revenue grew 18% YoY to $42.3M",
          "Policy count reached 1.2M active",
          "Claims ratio improved to 62%",
          "Customer satisfaction: 4.7/5.0"
        ]
      }
    }
  }
```

```
    ]
  }
```

8.3 Generation Record (MongoDB document)

```
  {
      "_id": "ObjectId",
      "template_id": "ObjectId ref",
      "prompt": "Quarterly business review for Good Sam RV Insurance",
      "slide_plan": { ... },
      "output_file_id": "GridFS ObjectId",
      "slides_generated": 12,
      "status": "completed",
      "retries_used": 0,
      "created_at": "2026-04-03T10:05:00Z"
  }
```

---

9. API Design

POST /api/templates/upload

Upload and auto-analyze a master template.

```
Request:  multipart/form-data { file: .pptx, name: string }
Response: 201 { template_id: str, name: str, status: "analyzing" }
```

Analysis runs as a BackgroundTask. Poll `GET /api/templates/{id}` for status.

GET /api/templates

List all profiled templates.

```
Response: 200 [
    {
        id: str,
        name: str,
        total_slides: int,
        usable_layouts: int,       # excludes guidance slides
        slide_types: ["title", "bullet", "chart_bar", ...],
        status: "ready" | "analyzing" | "failed",
```

```
        created_at: str
    }
]
```

GET /api/templates/{id}

Full template details including profile.

```
Response: 200 {
    id: str,
    name: str,
    profile: { ...full TemplateProfile... },
    status: str
}
```

POST /api/generate

Start PPT generation.

```
Request: {
    template_id: str,
    prompt: str
}
Response: 202 {
    generation_id: str,
    status: "processing"
}
```

Generation runs as a BackgroundTask. Track via SSE.

GET /api/generate/{id}/status (SSE)

Server-Sent Events stream for real-time progress.

```
event: progress
data: {"stage": "loading_template", "message": "Loading template
profile...", "progress": 10}

event: progress
data: {"stage": "planning_outline", "message": "Planning slide structure
(12 slides)...", "progress": 25}
```

```
event: progress
data: {"stage": "planning_content", "message": "Generating content for 12 slides...", "progress": 45}

event: progress
data: {"stage": "validating", "message": "Validating slide plan...", "progress": 60}

event: progress
data: {"stage": "rendering", "message": "Rendering slide 7 of 12...", "progress": 75}

event: complete
data: {"generation_id": "uuid", "download_url": "/api/generate/uuid/download", "slides": 12}

event: error
data: {"message": "Content planning failed after 2 retries", "stage": "planning_content"}
```

GET /api/generate/{id}/download

Download the final PPTX.

```
Response: 200 application/vnd.openxmlformats-officedocument.presentationml.presentation
Content-Disposition: attachment; filename="presentation.pptx"
```

---

10. User Journeys

Journey 1: Upload Template + Generate

```
User                                  System
 |                                      |
 ├─ Upload PPTX ──────────────────────► | Store in GridFS
 |                                      | Start BackgroundTask:
 |                                      |   parse_template (python-pptx)
 |                                      |   extract_guidance (Sonnet)
 |                                      |   save_template_profile (MongoDB)
 |  ◄── "Analyzing..." ──────────────── |
```

```
 |   ◀── "Template ready!" ──────── |
 |                                  |
 ├─ Enter prompt ──────────────────▶ |
 ├─ Click "Generate" ──────────────▶ | Start BackgroundTask:
 |                                  |
 |   ◀── SSE: "Planning (12 slides)" | Pipeline → plan_outline →
plan_content
 |   ◀── SSE: "Validating..." ────── | Pipeline → validate node
 |   ◀── SSE: "Rendering 5/12..." ── | Pipeline → render node
 |   ◀── SSE: "Done!" ────────────── | Pipeline → store in GridFS
 |                                  |
 ├─ Click "Download" ──────────────▶ | Stream PPTX from GridFS
 |                                  |
```

Journey 2: Select Existing Template + Generate

```
User                               System
 |                                  |
 ├─ Browse template library ───────▶ | GET /api/templates
 |   ◀── Template list ────────────── |
 |                                  |
 ├─ Select "CWGS Q1 2025" ─────────▶ |
 ├─ Enter prompt ──────────────────▶ |
 ├─ Click "Generate" ──────────────▶ | Load cached profile from MongoDB
 |                                  | (no re-analysis needed)
 |   ◀── SSE: progress ────────────── | ... same pipeline ...
 |   ◀── SSE: "Done!" ─────────────── |
 |                                  |
 ├─ Click "Download" ──────────────▶ | Stream PPTX
 |                                  |
```

---

11. Phase-by-Phase Implementation

Phase 1: Foundation (Week 1)

Goal: Project scaffolding, MongoDB setup, basic API skeleton.

- [ ] Initialize `backend/` folder structure as specified in Section 4
- [ ] Set up `pyproject.toml` with dependencies:

    - `fastapi`, `uvicorn`, `motor`, `python-pptx`, `lxml`, `langgraph>=1.1.4`, `langchain-core>=0.3`, `langchain-anthropic>=0.3`, `pydantic-settings`, `python-multipart`, `sse-starlette`
- [ ] Implement `config.py` with pydantic-settings (MONGODB_URI, ANTHROPIC_API_KEY, etc.)
- [ ] Implement `database.py` with Motor async client + GridFS buckets + lifespan hooks
- [ ] Implement `api/dependencies.py` (get_db, get_gridfs injectors)
- [ ] Implement `api/middleware.py` (CORS, error handler)
- [ ] Implement `api/routes/health.py` (GET /health → DB ping)
- [ ] Implement `core/exceptions.py` and `core/logging.py`
- [ ] Verify: `uvicorn app.main:app` starts, `/health` returns OK with MongoDB connected

Phase 2: Template Parser (Week 1-2)

Goal: Parse any uploaded PPTX and extract complete structural metadata.

- [ ] Implement `tools/template_parser.py`:
    - Extract all slides with placeholders (idx, type, name, position in EMUs, text styling)
    - Extract charts per slide (chart type, series count, categories, current values, colors)
    - Extract images per slide (position, size — flagged as DO NOT TOUCH)
    - Extract tables per slide (rows, cols, header, cell styling)
    - Extract theme from `ppt/theme/theme1.xml` via lxml (all colors + font scheme)
    - Estimate max_chars per placeholder from (width ÷ average_char_width_at_font_size)
    - Extract raw text content of every slide (for guidance detection)
- [ ] Implement `schemas/template_profile.py`:
    - Pydantic models: `TemplateProfile`, `SlideMetadata`, `Placeholder`, `ChartMeta`, `TableMeta`, `ImageMeta`, `ThemeColors`, `BrandRules`
- [ ] Write tests: parse a sample PPTX, verify extracted metadata matches expected values

Phase 3: Guidance Extractor + Template Upload API (Week 2-3)

Goal: Classify guidance vs content slides, extract brand rules, wire up the upload endpoint.

- [ ] Implement `tools/guidance_extractor.py`:
    - Send all slide text content to Claude Sonnet
    - Get back: classification per slide (guidance/content_layout) + brand_rules JSON
    - Filter guidance slides out of available_layouts
    - Group content_layout slides into type pools (title, bullet, chart_bar, etc.)
- [ ] Implement `graph/prompts/guidance_extractor.txt` — the system prompt for slide classification
- [ ] Implement `services/template_service.py`:

    - `upload_template()` : store PPTX in GridFS → parse → extract guidance → save profile to MongoDB
- ☐ Implement `api/routes/templates.py` :
    - `POST /upload` — accept PPTX, trigger BackgroundTask, return template_id
    - `GET /` — list all templates with status
    - `GET /{id}` — full template profile
- ☐ Implement `models/template.py` — request/response Pydantic models
- ☐ Write tests: upload a sample PPTX → verify profile stored in MongoDB with correct classification

Phase 4: Content Planner Agent (Week 3-4)

Goal: Given prompt + template profile, generate a complete, structured slide plan.

- ☐ Implement `graph/prompts/planner_outline.txt` :
    - Includes: available layout types, slide type pools, brand rules
    - LLM decides: slide count, narrative arc, layout selection per slide
- ☐ Implement `graph/prompts/planner_populate.txt` :
    - Includes: outline + full placeholder constraints + chart metadata per selected layout
    - LLM generates: text per placeholder (within max_chars), chart data (matching structure), table data
- ☐ Implement `schemas/slide_plan.py` :
    - Pydantic models: `SlidePlan` , `SlideOutline` , `SlideContent` , `ChartData`
    - JSON schema export for LLM structured output via `with_structured_output()`
- ☐ Implement `graph/nodes.py` — plan_outline_node and plan_content_node:
    - Two-phase call: outline → populate
    - Uses `init_chat_model()` for multi-model support
    - Claude Opus 4.6 for content planning
- ☐ Test with 5+ varied prompts against a real template profile
    - Verify: correct slide count decisions, valid layout selections, content within char limits

Phase 5: Validator (Week 4)

Goal: Deterministic Python validation of slide plans before rendering.

- ☐ Implement `tools/validator.py` :
    - Check every text field: `len(content) <= max_chars` for each placeholder
    - Chart data validation:
        - Series count matches template chart's series_count
        - Category count matches template chart's category_count

        - Pie chart values sum to 100
        - No null/negative values where not allowed
    - Table validation: row/col counts match template
    - Required fields: every placeholder in layout must have content
    - No duplicate layout variant usage (don't use slide index 23 twice)
    - Slide count in plan matches outline's `decided_slide_count`
    - Return: `{valid: bool, errors: [{slide_index, field, message}], warnings: [...]}`
- ☐ Write tests: craft valid + invalid slide plans, verify validator catches all issues

Phase 6: PPTX Renderer (Week 4-5)

Goal: Take validated slide plan + template → produce final PPTX.

- ☐ Implement `tools/renderer.py`:
    - Open original template PPTX from GridFS
    - Determine which slides to KEEP (only those in the slide plan)
    - Delete all other slides (guidance slides + unused layouts)
        - **Critical:** delete in reverse order to avoid index shifting
    - For each kept slide:
        - **Text placeholders:**
            - Locate shape by placeholder idx
            - Capture existing text formatting (font family, size, color, bold, italic) from first run
            - Clear existing text
            - Set new text preserving captured formatting
            - If text still too long: step down font size (floor: original_size × 0.7)
        - **Charts:**
            - Access chart via `shape.chart`
            - Replace data using `chart.replace_data(chart_data)` with values from plan
            - DO NOT touch chart type, colors, legend, gridlines
        - **Tables:**
            - Fill cells preserving existing cell formatting
        - **Images:**
            - DO NOT TOUCH
        - **All other shapes:**
            - DO NOT TOUCH
    - Clean up residual placeholder text ("INSERT TEXT HERE", "LOREM IPSUM", "DELETE BEFORE USE")
    - Save final PPTX bytes

- ☐ Implement `tools/storage.py`:
  - `store_generated_pptx()`: upload bytes to GridFS `generated_files` bucket
  - `retrieve_generated_pptx()`: download from GridFS
- ☐ Write tests: render a slide plan against a real template, verify PPTX opens correctly
  - Manual verification: open in PowerPoint/Google Slides, check formatting preserved

Phase 7: LangGraph Pipeline + Full Integration (Week 5-6)

Goal: Wire everything together with LangGraph StateGraph. Prompt in → PPTX out.

- ☐ Implement `graph/state.py`:
  - `PipelineState` TypedDict with all fields (template_id, prompt, profile, slide_outline, slide_plan, validation_result, retry_count, pptx_bytes, output_file_id, status, error)
- ☐ Implement `graph/nodes.py` — all node functions:
  - `load_profile_node`: async MongoDB read
  - `plan_outline_node`: Claude Opus via `init_chat_model()`, emit SSE progress
  - `plan_content_node`: Claude Opus via `init_chat_model()`, include validation errors on retry
  - `validate_node`: call deterministic validator
  - `render_node`: call python-pptx renderer
  - `store_node`: GridFS write, update generation record
- ☐ Implement `graph/pipeline.py`:
  - StateGraph definition with all nodes and edges
  - `validation_router` conditional edge: valid → render, invalid + retries < 2 → plan_content
  - Compile graph: `pipeline = builder.compile()`
- ☐ Implement `graph/prompts/planner_outline.txt` and `graph/prompts/planner_populate.txt`
- ☐ Implement `services/generation_service.py`:
  - `generate_ppt()`: invoke pipeline via `pipeline.ainvoke()`, track progress
  - `stream_generation()`: use `pipeline.astream()` with `stream_mode="custom"` for SSE
- ☐ Implement `api/routes/generate.py`:
  - `POST /generate` — trigger BackgroundTask
  - `GET /{id}/status` — SSE stream (using sse-starlette + LangGraph custom streaming)
  - `GET /{id}/download` — stream PPTX from GridFS
- ☐ Implement `models/generation.py` — request/response models
- ☐ End-to-end test: upload template → generate from prompt → download PPTX → verify in PowerPoint

Phase 8: Testing & Hardening (Week 6-7)

Goal: Comprehensive testing, error handling, logging.

- ☐ Write integration tests:
  - Upload template → verify profile created correctly
  - Generate with various prompts → verify PPTX output
  - Test retry logic: craft a prompt that causes initial validation failure
  - Test edge cases: very long prompt, very short prompt, empty prompt
- ☐ Add structured logging throughout pipeline:
  - Log: template_id, generation_id, stage, duration, token usage
- ☐ Add proper error handling:
  - Template parse failures → meaningful error returned to user
  - LLM failures → retry once, then return error
  - Render failures → log details, return error
- ☐ Add input validation:
  - Max file size for template upload (50MB)
  - Prompt length limits
  - Rate limiting on /generate endpoint

Phase 9: Next.js Frontend (Week 7-9)

Goal: Build the user-facing web application.

- ☐ Initialize Next.js project in `frontend/`
- ☐ Template Management pages:
  - Upload page: drag-and-drop PPTX, show analysis progress
  - Gallery page: grid of templates with name, slide count, usable layout count
- ☐ Generation page:
  - Template selector dropdown (or click from gallery)
  - Prompt textarea with guidance hints
  - "Generate" button → progress bar with SSE stages
  - Download button on completion
- ☐ Generation history page:
  - List of past generations with prompt preview, date, re-download

---

12. Cost Estimation

Per-Generation LLM Cost

Call	Model	Est. Input Tokens	Est. Output Tokens	Est. Cost
Content Planner Phase A (Outline)	Opus 4.6	~10,000 (profile + prompt)	~2,000 (outline)	~$0.25
Content Planner Phase B (Populate)	Opus 4.6	~15,000 (outline + constraints)	~5,000 (full plan)	~$0.45
Retry (if needed, ~30% of cases)	Opus 4.6	~18,000	~5,000	~$0.50
Total (no retry)				**~$0.70**
Total (with 1 retry)				**~$1.20**

Per-Template Analysis Cost (one-time)

Call	Model	Est. Cost
Guidance Extractor	Sonnet 4.6	~$0.05
Total		**~$0.05**

Optimization Levers

1. **Template profile cached**: Guidance extraction runs once per upload; all subsequent generations from that template skip it.
2. **LangGraph orchestration is free**: Pipeline routing is pure Python code — no LLM tokens spent on orchestration.
3. **Prompt caching**: Anthropic prompt caching on system prompts + template profile (reused across generations).
4. **Multi-model flexibility**: Can swap expensive Opus calls for cheaper models per node via config — no code changes.

---

13. Risks & Mitigations

Risk	Impact	Mitigation
LLM generates content exceeding placeholder limits	Text overflow in PPTX	Deterministic validator catches before rendering; retry Planner with specific char limits; auto-font-sizing as last resort in renderer
Template has unusual layouts the parser can't handle	Incomplete profile	Log unparseable elements; fall back to basic placeholder detection by idx; allow manual profile editing later

Risk	Impact	Mitigation
Chart data structure mismatch	Chart renders incorrectly	Parser extracts exact series/category count → Validator enforces exact match → Renderer uses `replace_data()` which preserves structure
Guidance slides not correctly identified	Brand rules missed or content layouts excluded	LLM classification + heuristic fallback (slides with >200 words of non-placeholder text are likely guidance)
LLM picks wrong number of slides for prompt	Too many/few slides	Planner prompt includes explicit heuristics (simple=5-8, standard=10-15, detailed=15-25); user can override in future phases
Slide deletion reorders indices	Wrong slides rendered	Delete in reverse index order; map by original index before any deletions
python-pptx can't handle animations/transitions	Missing animations in output	Documented limitation: master-level animations carry through, but per-slide animations are template-inherited only
Font not installed on server	Font substitution in rendered PPTX	python-pptx preserves font name in XML — it renders correctly when opened on a machine with the fonts installed. Document font requirements for users.
Concurrent generations overload the server	Timeouts, OOM	BackgroundTasks with a generation queue; max 3 concurrent generations for Phase 1; scale with Celery/Redis in later phases
Anthropic API rate limits	Generations fail	Exponential backoff retry; queue overflow → return "busy, try again"
LLM provider outage	Pipeline blocked	LangGraph multi-model: swap to alternative provider per node via config (e.g., Opus → GPT-4o for content planning)